

End-Evaluation Assessment Report

Design and Verification of a
High-Speed DDR Display Controller

Team ID: Elec-13

Team Members:

Harshit Gautam (250002031) Ayusman Dash (250002017)
Anshu Yadav (250002014) Yuvraj Diwakar (250002087)

Project Mentor:

Rucha Gadgil



Science & Technology Council, IIT Indore
Students' Gymkhana



Contents

1	Progress till Mid-Evaluation	1
1.1	Modules in the Design	1
1.2	Waveforms to showcase the modules' working	1
2	Implementation of DDR	2
2.1	Conceptualizing DDR:	2
2.2	Implementation of DDR in the project	3
2.3	A note on Clock Forwarding:	4
3	Video Generation prior to AXI Implementation	4
3.1	Simulation using ODDR Testbench	4
3.2	Using python for video generation	4
4	Bonus Implementation: Dynamic Control using AXI4-Lite	4
4.1	AXI4-Lite Slave Interface Implementation	4
4.2	AXI4-Lite Communication Channels	5
5	Creation of IP and Block Design	5
5.1	IP: myip	5
5.2	Block Design	5
6	Pin and Timing Constraints Overview	6
7	Synthesis, Implementation and Bitstream Generation	7
7.1	Design and Tool Overview	7
7.2	Timing Summary Metrics	7
7.3	Clock Architecture Summary	8
7.4	Utilization Summary	8
8	Interfacing FPGA with keyboard	8
8.1	Updates in myip	8
8.2	Using Vitis for dynamic inputting from terminal	9
9	Testbench for Verification	9
10	Conclusion	10

1 Progress till Mid-Evaluation

1.1 Modules in the Design

In the context of the progress made to mid-evaluation, the different modules mentioned below have been used in the design to implement the VGA variables' generation including the synchronization variables, the output color variables, etc.:

- *params.vh*: This is a verilog header file that has been used to initialize the various parameters in reference to VGA Standards in order to lay the foundation for video generation.
- *horizontal_count.v*: This module has been used to update the horizontal counter variable at every rising edge of the clock. It also contains the 'Enable' signal, which turns high every time a complete set of horizontal pixels is completely generated, which then tells the vertical counter to increment by one in the vertical_count module.

Input Variable: clk

Output Variables: hcount, enable

- *vertical_count.v*: This module is responsible for updating the vertical counter variable and the frame variable. The vertical counter variable is incremented by one each time the 'Enable' signal is high and the frame variable turns high after all the pixels have been completely generated.

Input Variables: enable, clk

Output Variables: vcount, frame

- *block.v*: This module has been used to create the pixel block by outputting the respective colors when the counters are within specific limits. Additionally, in this module, the block's movement along with its bouncing logic has been implemented.

Input Variable: clk

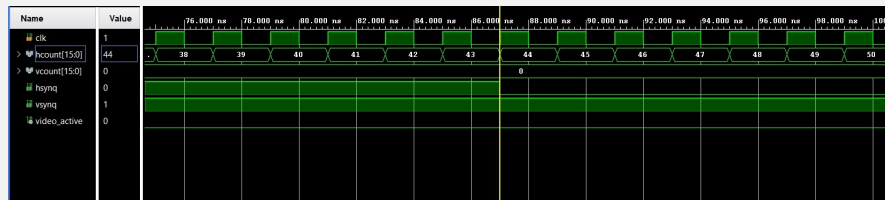
Output Variables: hsync, vsync, red, green, blue

Click to view the Design Source Files

1.2 Waveforms to showcase the modules' working



Bouncing Instance: The x direction changes to 0 (block bounces leftward) and the y direction also changes to 0 (block bounces upward).

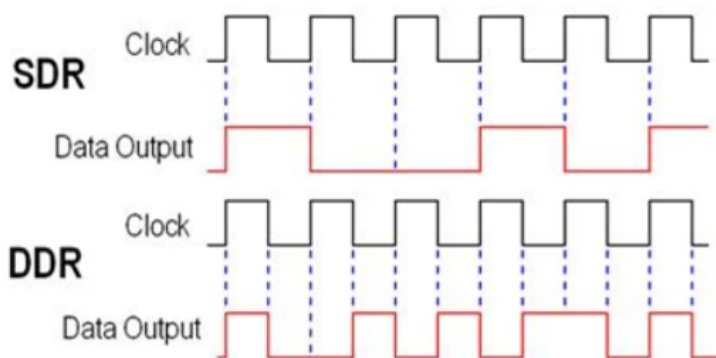


Synchronization Instance: The horizontal sync variable turns low.

2 Implementation of DDR

2.1 Conceptualizing DDR:

DDR is a technique which transmits double the data per clock cycle by utilizing the clock's rising edge (low to high) and the clock's falling edge (high to low).



(*There is a small clarification that needs to be made, which is that DDR does not alter the frequency or the like, but rather increases the throughput by taking double the amount of bits per clock cycle, which is highly effective for the high-speed demand. Hence, putting the double data output as "obtaining an effectively double frequency" is a logical inconsistency. To demonstrate numerically, say the input clock accepts data at 74.25Mbps . Then, DDR outputs a double data rate of $74.25 \times 2 = 148.5\text{Mbps}$.)

In the context of the project, the ODDR primitive provided by Xilinx has been used to utilize the concept of DDR. It is a part of the IO Block of most modern Xilinx FPGAs and is beneficial for the following points:

- **Bypasses Fabric Jitter:** Implementing DDR logic inside the general FPGA fabric using standard look-up tables (LUTs) introduces routing delays and clock jitter. This corrupts timing at high speeds.
- **Guarantees Precision:** The ODDR primitive is a dedicated, hard-wired silicon circuit right at the physical pin. It guarantees a perfect 50/50 duty cycle split between the two data streams.
- **SDR-to-DDR Conversion:** It takes two parallel SDR data streams from the internal FPGA fabric and interleaves them into a single external DDR data stream.

2.2 Implementation of DDR in the project

Utilizing DDR requires updating the modules, primarily the block module, to output two synchronization values and two color values in correspondence to each color since each clock cycle now takes up two values contrary to the SDR domain. Hence, a new module was created, namely 'DDR_display', to generate the data. Additionally, it should be noted that the final output variables are single-bus values, i.e. two values of each parameter (say [3:0]red1 and [3:0]red2) are clubbed into a single value (therein, [3:0]ddr_red).

The module (ddr_display.v) parameters are mentioned below:

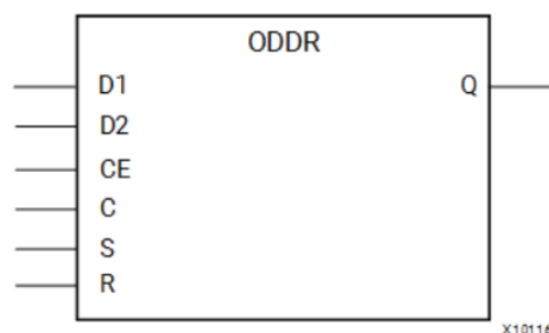
Input Variables: clk_half, i_box_size, i_box_speed, state

Output Variables: ddr_hsynq, ddr_vsynq, ddr_red, ddr_blue, ddr_green

Additionally, the main block module also details some changes apart from the two output variables, most notably the fact that instead of incrementing the horizontal counter by one per positive edge of the clock, it is incremented by **two**. This is important since DDR requires two bits of information per clock cycle, which is provided by the aforementioned reform. Also, this increment helps in generating two variables per horizontal counter (e.g. red1 corresponding to hcount1 and red2 corresponding to hcount2).

```
wire [15:0]hcount1 = hcount;
wire [15:0]hcount2 = hcount+1;
hcount = hcount+2
```

Primitive: Dedicated Dual Data Rate (DDR) Output Register



While using ODDR, the user generally has to initialize the primitive with the configurations of whether data is accepted on the same clock edge (*SAME_EDGE*), which is the preferred way for timing closure reasons or on both clock edges (*OPPOSITE_EDGE*), initialization value (which is set to **0** for our case), and *SRTYPE* (which is set to *SYNC*), which is to determine if the set and reset are synchronous with the clock. In the context of using ODDR, mentioned below are its parameters and a brief explanation of each of them:

- **C:** This is an *input pin* to accept the clock.

- **CE:** This is an *input pin* that, when asserted high, enables the clock input on port C.
- **D1/D2:** These are *input pins* into which DDR data is presented in the ODDR module.
- **R/S:** These are *input pins* that reset/set according to the SR Type set during primitive initialization.
- **Q:** This is the *output pin* that connects ODDR output to the IO Pad.

2.3 A note on Clock Forwarding:

As the name suggests, *Clock Forwarding* is a technique that is used to forward the clock signal to an externally connected device such as a DAC, by using the ODDR primitive in a clever way, wherein the D1 and D2 input pins are hard-coded to 1 and 0 respectively, which is then provided to the external device as an input. This is an exact replica of the original clock. This method saves resources and also mitigates noise-induced error. Hence, to summarize, Clock forwarding is needed to synchronize data transmission between separate chips or modules by sending a dedicated clock signal alongside the data. Without it, circuits suffer from severe skew, timing violations, and data corruption.

3 Video Generation prior to AXI Implementation

3.1 Simulation using ODDR Testbench

A testbench was created to simulate the ODDR primitive, which created an uut (unit under test) for the ddr_display module and ran the simulation for a total of 600 frames, compiling the output color sequence into a .hex file.

3.2 Using python for video generation

The aforementioned testbench generated a .hex file (entailing a sequence of RGB combinations in xxxxxx format, e.g. 0000FF), pertaining to 600 frames, which was later compiled into a video using a python script. Alternatively, ffmpeg (a processing engine for format conversion and editing) can also be used, although it requires an .ppm file (i.e. 600 .ppm file images, then combining them into single video), contrary to the previous .hex file.

[Click to view the new Design Source Files](#)

4 Bonus Implementation: Dynamic Control using AXI4-Lite

4.1 AXI4-Lite Slave Interface Implementation

To make the DDR Display Controller configurable during runtime, a custom AXI4-Lite slave IP was created in Vivado and connected to the MicroBlaze processor. Instead of using fixed values inside the Verilog code, the block colour, size and speed are now

controlled through memory-mapped registers. Whenever the processor writes new values to these registers, the display controller immediately updates the corresponding parameters without requiring the FPGA to be reprogrammed.

The implemented AXI interface provides:

- Memory-mapped control registers.
- Dynamic control of block's colour, size and speed.
- A fully synthesizable hardware.

4.2 AXI4-Lite Communication Channels

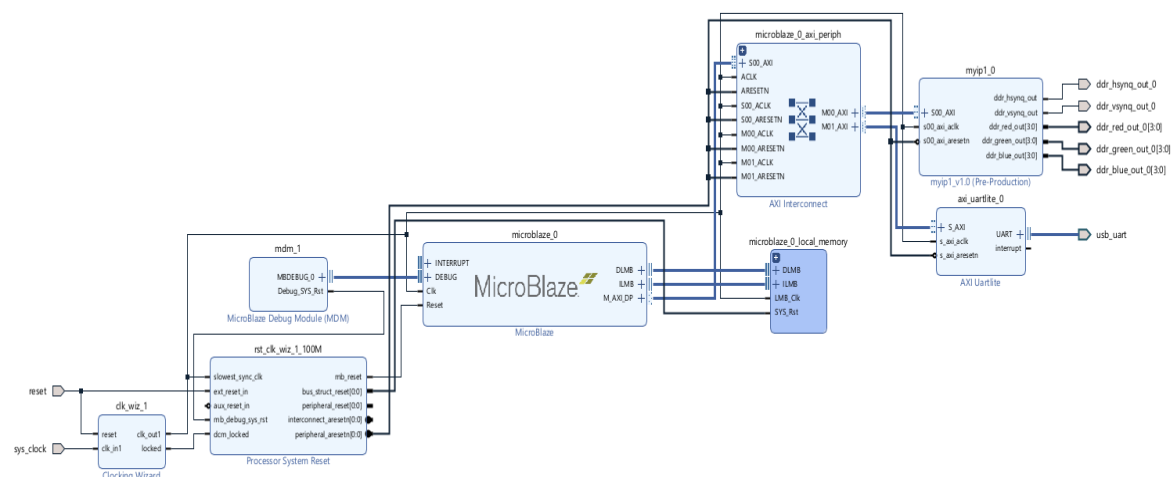
The AXI4-Lite protocol transfers data using five independent communication channels. The **AW** and **W** channels carry the write address and write data, while the **B** channel confirms that the write operation has been completed successfully. Similarly, the **AR** channel specifies the register to be read, and the **R** channel returns the requested data. All these channels use the **VALID-READY** handshake to ensure reliable communication between the processor and the custom IP.

5 Creation of IP and Block Design

5.1 IP: myip

Creating a custom IP (Intellectual Property) in Xilinx Vivado allows for code reusability, modular system design, and standardized bus integration. It transforms raw HDL code into plug-and-play components. Hence, to put the code reusability aspect into good use, the original verilog design (i.e. ddr_display) was first coupled up with the AXI interface and wrapped (essentially, an HDL wrapper was created so that the input and output pins for the design are easier to accentuate). The file henceforth generated was component.xml.

5.2 Block Design



The above block diagram shows the overall hardware architecture of the system built around the MicroBlaze soft processor. A clocking wizard generates the required internal clock (74.25MHz) from the external system clock (100MHz), while the processor system reset block ensures stable startup and proper reset synchronization. The MicroBlaze processor is connected to local memory through the ILMB/DLMB interfaces for fast instruction and data access, and it also communicates with peripheral devices through the AXI interconnect.

The design includes the MicroBlaze Debug Module (MDM) for debugging and monitoring, and an AXI UARTLite peripheral for serial communication through the USB UART interface. In addition, the custom peripheral myip1_0 appears to control video/display-related output signals such as synchronization and RGB data lines. Overall, the diagram illustrates a self-contained embedded system in which the processor, memory, reset circuitry, communication interface, and custom output peripheral are cleanly integrated on a single FPGA device.

6 Pin and Timing Constraints Overview

The constraints file was set-up considering the Basys3 FPGA development board provided by Xilinx which is especially proficient at VGA interfacing as it has dedicated pins to do the same.

Clock and Reset

- **System Clock (sys_clock):** Assigned to pin **W5** with **LVC MOS33** I/O standard. A 100 MHz clock is defined (`period = 10 ns`, 50% duty cycle) using `create_clock`, enabling accurate timing analysis and synthesis optimization.
- **Reset (reset):** Assigned to pin **U18**, also **LVC MOS33**. Marked as a false path since it is an asynchronous input not required to meet normal setup/hold timing.

VGA Output Signals

- **Sync Signals:**
 - Horizontal sync (`ddr_hsynq_out_0`) → pin **P19**, **LVC MOS33**
 - Vertical sync (`ddr_vsynq_out_0`) → pin **R19**, **LVC MOS33**
- **Color Signals (4-bit per channel, top 4 bits of 8-bit buses):**
 - **Red[3:0]:** N19, J19, H19, G19
 - **Green[3:0]:** D17, G17, H17, J17
 - **Blue[3:0]:** J18, K18, L18, N18

All use **LVC MOS33**, matching the board's 3.3 V I/O and the VGA monitor's DAC inputs.

USB UART (AXI Uartlite Interface)

- **TX** (usb_uart_txd): Pin **A18**, **LVC MOS33**
- **RX** (usb_uart_rxd): Pin **B18**, **LVC MOS33**

The RX path is marked as a false path, as it is an asynchronous external input.

Timing Exceptions

- The clock `clk_out1_design_1_clk_wiz_1_1` (from the Clocking Wizard) is declared **asynchronous** to other clocks using `set_clock_groups`, preventing unnecessary cross-domain timing checks.
- False paths are set on `reset` and `usb_uart_rxd` to exclude asynchronous inputs from static timing analysis.

Together, these constraints ensure correct pin mapping, I/O voltage levels, and timing behavior for reliable VGA signal generation and board-level interfacing.

7 Synthesis, Implementation and Bitstream Generation

After successfully creating a block design, the design was synthesized to check if the design can be realized on hardware using the constraints file, implemented to map components onto the FPGA and at last, the bitstream was generated which can be directly loaded on the FPGA.

7.1 Design and Tool Overview

- **Tool Version:** Vivado v.2023.2 (64-bit build 4029153)
- **Design Name:** `design_1_wrapper`
- **Target Device:** Basys-3 (Artix-7 Family) (`7a35t-cpg236-1`)
- **Design State:** Routed (Post-Implementation)
- **Analysis Corners Analyzed:** Multi-Corner Analysis (Slow and Fast Process Corners enabled with pessimism removal)

7.2 Timing Summary Metrics

The design timing summary confirms that all user-specified timing constraints have been completely met with positive slack values across all endpoints.

Table 1: Design Timing Summary Overview

Metric	WNS (ns)	TNS (ns)	Failing Endpoints	Total Endpoints
Setup Analysis	4.721	0.000	0	4,835
Hold Analysis	0.091	0.000	0	4,835
Pulse Width	3.000	0.000	0	1,843

Key observations from Table 1:

- **Worst Negative Slack (WNS):** 4.721 ns, indicating that the design comfortably meets its setup time constraints with a substantial safety margin.
- **Total Negative Slack (TNS):** 0.000 ns, proving that there are zero setup violations across all 4,835 synchronous endpoints.
- **Worst Hold Slack (WHS):** 0.091 ns, confirming proper hold-time margins with no race conditions or hold violations (THS = 0.000 ns).

7.3 Clock Architecture Summary

The design incorporates multiple clock domains managed via clock management tiles (MMCM/PLL) and internal debugging wrappers:

- **System Clocks:** `sys_clock` / `sys_clk_pin` operating at 100.000 MHz (10.000 ns period).
- **Derived Clocks:** `clk_out1_design_1_clk_wiz_1_1` operating at 74.250 MHz (13.468 ns period).

7.4 Utilization Summary

The post-implementation device utilization report provides the resource breakdown for the target Basys-3 device (xc7a35tcpg236-1), detailing the consumption of logic cells, memory blocks, and I/O resources based on the fully placed design:

Table 2: Post-Implementation Device Utilization Summary

Resource	Used	Available	Utilization (%)
Slice LUTs	1,790	20,800	8.61%
Slice Registers	1,568	41,600	3.77%
Block RAM Tile	4	50	8.00%
DSPs	0	90	0.00%
Bonded IOB	18	106	16.98%

Key takeaways from the placement utilization data include:

- **Logic Utilization:** The design consumes 1,790 Look-Up Tables (LUTs) out of 20,800 available, maintaining a low footprint of 8.61%.
- **Registers:** A total of 1,568 flip-flops/registers are utilized out of 41,600, corresponding to 3.77% utilization.
- **Memory:** Block RAM utilization stands at 8.00% using 4 tiles.

8 Interfacing FPGA with keyboard

8.1 Updates in myip

Since the AXI protocol was implemented and for the ease and dexterity of the user, the **vitis** software was utilized in order to configure a few keys of the keyboard to send

signals to the FPGA utilizing the AXI channels. In order to do that, the myip IP was updated to change the previously hard-coded registers, i.e. `block_size`, `block_speed` and `block_color` into variables that can be altered at user's will. Mentioned below are the respective inputs and their corresponding changes in the associated variables:

- **cmd_write_pulse:** This register turns high once an input is detected from the user's end and enables the logic to act to update the state variables.
- **internal_box_color:** This register changes the box's color by hard-coding the color's ASCII values to a number (e.g., `8'h72` which is ASCII for R, i.e. red is mapped to 0). Then, this is coded in the verilog logic so as to update the color output to the respective color for the input received from the UART receiver.
- **internal_box_speed:** This register works similarly to the `internal_box_color` register, difference being the new ASCII codes of W and S being used to increase or decrease the speed by 1 pixel/posedge respectively.
- **internal_box_size:** This register also works similarly to the prior two registers, associating the ASCII codes of + and - with increasing or decreasing the size by 10 pixels, respectively.

8.2 Using Vitis for dynamic inputting from terminal

To input commands from keyboard, the Vitis terminal was used. **Vitis** is a software by Xilinx which is generally used to develop embedded software, AI inferences, and accelerated applications for Adaptive SoCs, FPGAs, and CPU targets. It replaces legacy Xilinx SDK workflows by enabling software development in standard languages like C and C++ without requiring low-level hardware or RTL expertise.

The code to parse the user input from the keyboard into signals that are transmitted via the AXI-UART protocol interface was written in C. The code was configured based on the bitstream generated from the verilog design, which used the AXI `address_write` and `data_write` functions to update the data at the necessary variable addresses according to the user's input.

9 Testbench for Verification

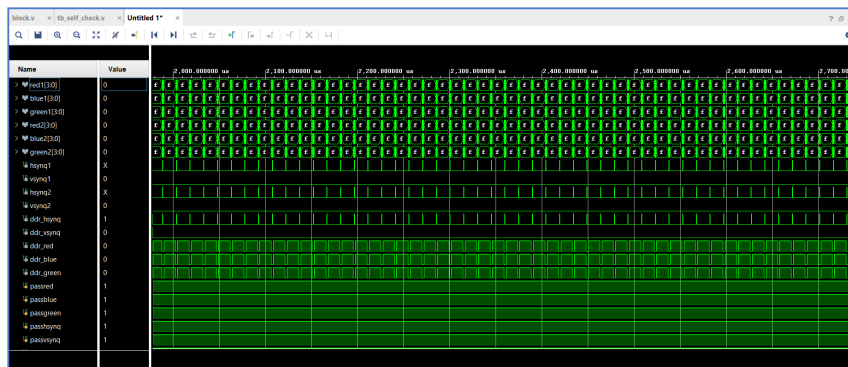


Figure 1: Verification of ODDR

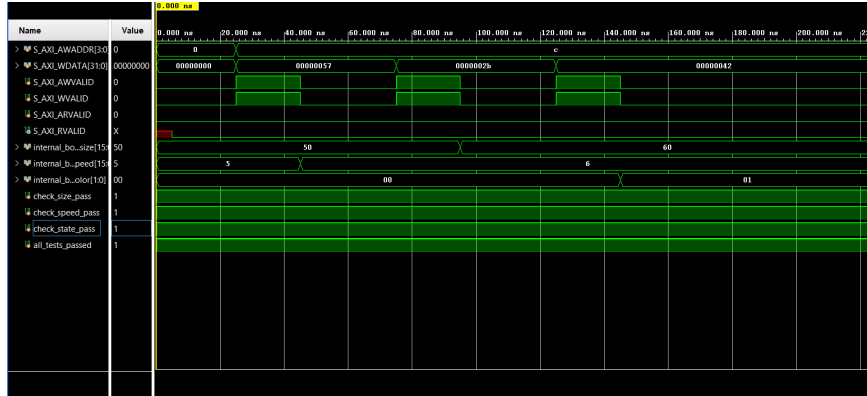


Figure 2: Verification of AXI

- **DDR Verification**, as shown in 1, is produced through a self-checking testbench in which double-sized data (e.g. red1[3:0], red2[3:0]) are passed through 'ODDR' block and then verified if its being successfully passed to the output (ddr_red_out_0). Half of the data is passed on the rising edge, while the other half is passed on the falling edge.

The pass variable for each color (e.g. passedred) compares the input value with the ODDR output value by processing the logic through respective instantiations of **ddr_display** and **block1** modules.

- **AXI Verification**, as shown in 2, is also produced through a self-checking testbench, that verifies the state variables such as internal_box_size, internal_box_color, etc. by comparing them with instantiated values from myip1_0.

This testbench compares the data provided through the AXI function calls with the internal variables and dynamically updates the respective pass variables, which remain high throughout and turn low only for a mismatch, verifying the integrity of the AXI interface.

10 Conclusion

In conclusion, this project successfully demonstrates the robust design and hardware verification of a high-speed DDR display controller, using the ODDR primitive to effectively double data throughput keeping an optimum base clock frequency. Furthermore, integrating a custom AXI4-Lite peripheral with a MicroBlaze processor allowed for seamless, dynamic runtime configuration of display parameters directly via user terminal inputs. Ultimately, the fully synthesized system comfortably met all timing and resource constraints on the target Basys-3 FPGA, validating its efficiency and reliability for real-time video generation.

[Click to access GitHub repository](#)